

LangChain Agent, Explained Simply

Same code as before — this time in plain, everyday language

1. What is this program actually doing?

Think of it like this: you're hiring an AI assistant, giving it exactly one tool ("search the internet"), and telling it two rules — don't search more than 3 times, and don't just ramble your answer, fill in a fixed form instead.

Both versions of the code do the exact same job. The only thing that changes between them is how the "search the internet" tool gets built. Step by step, the program:

- Loads an AI model (LLaMA 3.3, running on Groq's servers) — this is the "brain".
- Gives that brain one tool: search the web, using a service called Tavily.
- Connects the brain + the tool together into an **agent** — a loop where the brain can search, look at what it found, and decide to search again if it needs to.
- Forces the final answer into a fixed shape — an answer, plus a list of source links — instead of a free-flowing paragraph.
- Tells the brain "search at most 3 times", then asks it: find 3 AI Engineer internships on LinkedIn, with links.

So: **Version 1** builds the search tool by hand. **Version 2** uses a ready-made one. Everything else in the file is identical.

2. BaseModel — a form template for your data

BaseModel is basically a form template. Whenever you want to control exactly what shape a piece of data must have, you create a class that inherits from it.

```
class Source(BaseModel):  
    """Schema for a source used by the agent."""  
    url: str = Field(description="The URL of the source")
```

This says: create a form called Source. It has exactly one box on it — url — and whatever goes in that box must be text.

Because it inherits from BaseModel, three things happen automatically, for free:

- It checks the data is actually correct (a number can't sneak into a text box).
- It converts raw data (like a JSON reply from the AI) into a proper, usable Python object.
- It raises a clear error the moment something doesn't fit the form.

If you tried to hand it the wrong kind of data, you'd get an error straight away instead of a silent bug later:

✓ `Source(url="https://python.org")` — works, url is text.

✗ `Source(url=123)` — fails, 123 is a number, not text.

3. Field() — attaching a note to each box on the form

Field() lets you attach extra instructions to a box on the form — mainly a description of what should go in it.

```
url: str = Field(description="The URL of the source")
```

Here's the part that surprises most people:

The text inside `description=` isn't just a comment for humans reading the code. It gets shown directly to the AI. A clearer description genuinely helps the AI fill in that box correctly.

4. The two forms this project actually uses

There are two forms (schemas) in this code, and one sits inside the other.

Form 1 — **Source**: one box, `url` (must be text).

```
class Source(BaseModel):  
    url: str = Field(description="The URL of the source")
```

Form 2 — **AgentResponse**: what the AI's final reply must look like.

```
class AgentResponse(BaseModel):  
    answer: str = Field(description="The answer generated by the agent")  
    sources: List[Source] = Field(default_factory=list, description="...")
```

Breaking that second box down:

`List[Source]` → means: `sources` must be a list, and every single item inside that list must itself be a proper `Source` form — not just a plain piece of text.

✓ `sources = [Source(url="https://python.org"), Source(url="https://docs.python.org")]`

✗ `sources = ["python.org", "docs.python.org"]` — rejected, these are plain strings, not `Source` objects.

`default_factory=list` → means: "if nobody gives me any sources, just start with an empty list."

Why not just write `default=[]` instead? Because of a well-known Python trap: a single `[]` written as a default gets silently shared and reused across every `AgentResponse` you ever create — change one, and you'd accidentally change them all. `default_factory=list` sidesteps this by building a brand-new, separate list every single time.

So this works fine even with zero sources:

```
AgentResponse(answer="Hello")
```

5. Why go through all this trouble?

Because this `AgentResponse` form is handed to the agent like this:

```
create_agent(..., response_format=AgentResponse)
```

This tells the agent: "don't just reply with a paragraph — fill in this exact form." So instead of digging through a wall of text to find the answer, your code can just read `result.answer` and `result.sources` directly.

6. The imports — what each one is for, in one line

from typing import List

Just lets you say "this must be a list of `Source` objects" — pure labelling, doesn't change how the code runs.

from pydantic import BaseModel, Field

`BaseModel` = the form template. `Field` = the sticky note you attach to a box on that form.

from dotenv import load_dotenv

Loads your secret API keys from a `.env` file so you never type them directly into the code.

from langchain.agents import create_agent

One function that builds the whole "brain + tool + loop" agent for you, so you don't wire it by hand.

from langchain.tools import tool

The `@tool` decorator — turns a normal Python function into something the AI is allowed to call. Used only in Version 1.

from langchain_core.messages import HumanMessage, SystemMessage

Little labelled envelopes for text — one marked "instructions for the AI", one marked "the user's actual question".

from langchain_groq import ChatGroq

The plug that connects `LangChain` to `Groq`'s AI models.

from tavily import TavilyClient (Version 1 only)

The bare-bones, no-frills way to call the `Tavily` search API directly.

from langchain_tavily import TavilySearch (Version 2 only)

A ready-made search tool — already built, already wired up, nothing to assemble.

7. `load_dotenv()` — what's the point of this one line?

Picture a small file on your computer called `.env` that just contains your passwords:

```
GROQ_API_KEY=xxxxxxxxx
TAVILY_API_KEY=yyyyyyyyy
```

`load_dotenv()` reads that file and quietly makes those values available to the rest of the program. That's the only reason you can write `ChatGroq(model="...")` with no key typed anywhere — the key gets picked up automatically in the background.

This is why `load_dotenv()` always sits near the very top of the file — it has to run before anything that needs those keys.

8. Version 1 — building the search tool yourself

This version builds the search capability from scratch, piece by piece.

```
tavily = TavilyClient()

@tool
def search(query: str) -> str:
    """
    Tool that searches over internet
    Args:
        query: The query to search for
    Returns:
        The search results
    """
    print(f"Searching for: {query}")
    return tavily.search(query=query)

tools = [search]
```

Line by line, in plain words:

- `TavilyClient()` → makes one reusable connection to the Tavily search service. Built once, reused every time.
- `@tool` → the magic line. It looks at the function underneath it — its name, its explanation (the docstring), and its inputs — and turns it into something the AI is allowed to call by name. The AI never sees your Python code, only the name and the explanation.
- `def search(query: str) -> str:` → says this tool takes exactly one input, a piece of text called query.
- The triple-quoted explanation inside the function → isn't just a comment for humans. It's copied straight into the tool's description, which is literally what the AI reads to decide "should I use this tool right now?"
- `print(f"Searching for: {query}")` → just prints to your screen so you can watch what the AI is searching for. Doesn't affect the logic at all.
- `return tavily.search(query=query)` → actually runs the search and hands the results back to the AI.

Why not just call `tavily.search()` straight from your main code? Because the AI needs to decide for itself, mid-conversation, whether it even needs to search — and it can only make that decision if the ability to search is packaged as a named, described tool it can choose to use. Building it by hand like this also means you're free to add your own logging, caching, or tweaks later.

9. Version 2 — using a tool someone already built

Same job, done the fast way. Someone at LangChain already packaged Tavily search as a ready-to-use tool.

```
tools = [TavilySearch()]
```

That's it — one line does the job of everything in Version 1's ~13 lines. `TavilySearch()` already comes with its own name, its own explanation, and it already knows how to call the Tavily API.

One small catch worth knowing about:

The instructions given to the AI say "you only have access to one tool: `search`". That's true in Version 1, where the tool really is named `search`. But in Version 2, the ready-made tool is actually named `tavily_search`, not `search`. It still works fine here because the AI only has one tool to pick from anyway — but it's a mismatch worth knowing about if something ever goes wrong.

10. `create_agent(...)` — what does this actually build?

```
agent = create_agent(
    model=llm,
    tools=tools,
    response_format=AgentResponse
)
```

Three things you're handing over here:

- `model=llm` → the brain doing the thinking.
- `tools=tools` → the toolbox it's allowed to reach into (just the one search tool, here).
- `response_format=AgentResponse` → the exact form it has to fill in at the end.

What `create_agent` hands you back is a whole ready-to-run loop, built for you automatically. In plain English, that loop is:

1. Send the messages to the AI.
2. Did it ask to use a tool? If yes, run the tool and hand the result back to it.
3. Repeat step 1 with the new information.
4. Once it stops asking for tools, make it fill in the final form.

You never write this loop yourself — `create_agent` builds and runs it for you.

11. `main()` — what happens when you actually run this

- `SystemMessage(...)` → the ground rules, given before the conversation even starts. Here: "you may search at most 3 times, then you must answer with whatever you have."
- `HumanMessage(...)` → the actual question — find 3 AI Engineer internships on LinkedIn with links.
- `agent.invoke({...})` → presses play. Runs the whole loop from start to finish and waits for the final result.
- `print(result)` → dumps everything the AI did — every search, every result, and the final filled-in form — onto your screen. In real use, you'd usually only care about `result["structured_response"]`.

12. How it all fits together

The flow is exactly the same for both versions — the only box that's built differently is the toolbox itself.

- You ask a question → the AI reads it and the ground rules.
- The AI decides: do I need to search? → if yes, it calls the search tool.
- The search result comes back → the AI reads it and decides again.
- After at most 3 searches (or once it's satisfied) → it fills in the final form and stops.

13. Version 1 vs Version 2 — side by side

Aspect	Version 1 (TavilyClient, by hand)	Version 2 (TavilySearch, ready-made)
How the tool is built	Manual — you write the function yourself	Automatic — one constructor call
Lines of code needed	About 13 lines	1 line
Name the AI sees	"search" (whatever you named it)	"tavily_search" (fixed name)
How much control you have	Full — rename it, reshape results, add logging	Limited — only what the constructor allows
Extra package needed	tavily	langchain_tavily
Matches the system prompt's wording?	Yes, tool is really named "search"	Slight mismatch — real name is "tavily_search"
Best for	Learning how tools work under the hood	Shipping something quickly, with less to maintain

14. The one-paragraph takeaway

Both files do the exact same job — same brain, same form, same rules, same question. The only difference is where the search tool comes from.

- Version 1 shows you the mechanics — how a plain Python function becomes something the AI can call, using `@tool`.
- Version 2 shows you the shortcut — the same ability, already packaged, in exchange for a little less control.

Neither one is "more correct." Use Version 1 when you're learning or need to customise things. Use Version 2 when you just want it working, fast.